

CS-57 HUMAN–AI DATA ANNOTATION, PROVENANCE, AND DATASET EVALUATION PLATFORM

Project Progress



THE UNIVERSITY OF
SYDNEY

Group Based 5703 Capstone Project

Group Members

1. Michael Max (540559785)
 2. Parth (550620451)
 3. Jingwei Lin (520372766)
 4. Hanchen Wang (520374346)
 5. Yi Geng (520373453)
 6. Dishank Aswal (540870299)
 7. Kanishka Kathait (540870325)
 8. Tok Tin (Tim) Chung (550512950)
-

TABLE OF CONTENTS

Table of Contents	i
1. PROGRESS & ACHIEVEMENTS	1
1.1 The project in brief.....	1
1.2 Summary: are we on track?.....	2
1.3 Who the platform is for.....	3
1.4 From user needs to sprints	3
1.5 Sprint by sprint: what was built and what was solved	5
1.6 Front end: what each user can now do	6
1.6.1 Administrator and invitee	6
1.6.2 Project manager and task owner	7
1.6.3 Annotator.....	8
1.6.4 Reviewer.....	9
1.6.5 Expert	10
1.7 Back end and data	11
1.7.1 The dataset.....	11
1.7.2 AI: external models, not trained by us	12
1.7.3 How the back end supports the front end.....	12
1.7.4 Back-end scripts and services used in operations	12
1.7.5 Records, quality and inherited defects	13
1.8 Deliverables and the MVP	15
1.8.1 The MVP: complete, tested and working	16
1.9 Progress by epic	16
1.10 How we work.....	17
2. OBSTACLES	18
2.1 Obstacles met so far.....	19
2.2 Challenges by sprint and story	20
2.3 Risks ahead	21
2.4 Dependencies	22
2.5 Lessons learned.....	23
3. DEVIATION TO TIMELINE	24
3.1 The baseline	24
3.2 What moved, and why	24
3.3 Carry-over between sprints	26
3.4 Impact on the project	26
4. MILESTONES & REPORTING.....	27
4.1 Updated milestones.....	27
4.2 Remaining work, dependencies and the critical path.....	28
4.3 Who carries what	30
4.4 Slack, fallbacks and contingency	30
4.5 Reporting and communication	32

5. ACKNOWLEDGEMENT OF AI USE	32
APPENDIX A. IDENTIFIERS AND TERMS.....	32

1. PROGRESS & ACHIEVEMENTS

This report records team CS-57's progress at the start of Week 9 (8 October 2026). It concentrates on what has been done **since our Week 5 proposal**, that is, in the four sprints from 7 September to 7 October, and on the plan to the end of semester. Section 1 starts from the people who use the platform and follows each of their needs through the tickets and sprints that met it, then describes the front end, the back end and the data, and shows that the minimum viable product works. Section 2 covers the obstacles met, sprint by sprint, and the risks ahead; Section 3 where the timeline has moved and why; Section 4 the updated plan, milestones and reporting. Section 5 acknowledges the use of AI tools, and Appendix A explains the identifiers used throughout. Unless stated otherwise, every statement about the software refers to the main branch of the team's GitHub repository (USYD-CS-Capstone/hej) at commit 1dcbcdf of 7 October 2026.

1.1 The project in brief

Hej is Arc Intelligence's platform for producing labelled datasets for AI. People annotate items (for example, assigning a topic to a news headline), reviewers accept or reject that work, and disagreements are settled by an expert. The client handed over a working but immature codebase, a Next.js web application and a FastAPI service with a relational database, and asked us not to make labelling faster but to make the resulting dataset **defensible**: for every item in a released dataset the platform must be able to answer *how did this label get here*: who annotated it, what the AI suggested and whether the person accepted it, who reviewed it, how any disagreement was settled and under which guideline. It must also refuse to release data that cannot answer. The brief groups this into six pillars: **Govern** (roles and policies), **Assist** (AI suggestions), **Record** (provenance), **Adjudicate** (review and disputes), **Release** (gated, immutable releases) and **Evaluate** (repeatable measurement). Text annotation is the committed demonstration path; image, audio and video must keep working but are not extended.

How progress is measured. The scope is a backlog of user stories grouped into eleven epics, A to K (for example story D5, *finalised work cannot be overwritten*). Each story is broken into Jira tickets (SCRUM-n), and each ticket is delivered through a GitHub pull request (#n) that a team member other than the author reviews before it is merged. Work is estimated in units (u): one unit is two to three subtasks, and one

Jira story point equals one unit. A story counts as complete only when it meets our Definition of Done: demonstrable in the running application, shipped with tests, any linked defect proven fixed by a test that fails on the old code, documentation updated, and reviewed by someone other than its author.

1.2 Summary: are we on track?

In short, yes for the end-of-semester deliverables, with one critical path that needs watching:

- **The MVP is complete, tested and working.** The core workflow agreed with the client at Status Check 1 runs end to end on the client's text data, from inviting members to exporting finalised items, and is covered by automated and end-to-end tests that all pass on main (Section 1.8).
- **On schedule overall.** Build work ends on 28 October (Week 11) as planned; the remaining days go to the technical report, decision records and demonstration. No proposal deliverable has been dropped.
- **Ahead of plan:** the core annotation workflow, AI assistance and governance started one to two weeks earlier than the proposal scheduled, on top of engineering foundations delivered on time (Section 3.2).
- **About one week behind plan, and protected:** provenance and release, which extend the MVP to the full brief. This is the critical path; the replan of 1 October moved the provenance record from Week 11 to Week 9 so the release gate can be built on it (Sections 3.4 and 4.2).
- **Recovered:** evaluation was at zero at Status Check 2 (23 September); the harness is now merged and the casebook is in progress.
- **Waiting on the client:** the dispute record (stories E3/E4) has been paused since 3 October on an open product question (obstacle O4, risk R2).

Table 1. Progress in numbers, 7–8 October 2026

Measure	Value	Context
User stories complete	28 of 57 in scope (49%); 15 in progress; 14 not started	63 stories in the backlog; 6 set aside by decision (Section 1.9)
Pull requests merged into main	49 since 5 September; 22 in the last two weeks (#29–#57)	Each reviewed by a member other than its author
Automated tests on main, re-run on 8 October	Backend: 1,283 passed, 9 skipped, 0 failed (plus 207 subtests). Frontend: 400 passed. Evaluation harness: 4 of 4 cases pass	452 backend and 205 frontend tests at Status Check 2 (23 Sep); CI runs the backend suite on SQLite and PostgreSQL on every push
Inherited defects (audit of 30)	20 fixed; 6 fixed with closure check pending; 4 open	Critical: 3 of 6 fixed, 1 pending check, 2 scheduled for Week 9
Client decisions	7 initial questions and 15 more in two written rounds answered; 2 open	Rounds of 21 and 24 September; open questions in GitHub issues #38 and #40
Sprint delivery	77–93% of planned load completed in each sprint	Shortfall is work still in review at sprint end (Sections 1.5, 3.3)

1.3 Who the platform is for

Hej serves six kinds of user inside a client organisation (Table 2). Each holds one or more roles, and every action they take is checked on the server against those roles. The client, Arc Intelligence, owns the platform and decides product questions; it is a stakeholder rather than a user. Every story in the backlog serves at least one of these users, and the rest of Section 1 follows their needs.

Table 2. Target users, what they need, and the epics that serve them

User (role)	Who they are	What they need from Hej	Epics
Administrator	Runs an organisation's membership	Invite people with the right roles; change or remove roles; keep each project's work inside it	G Roles; J Organisations
Invitee	A person joining an organisation	See pending invitations and accept one, gaining exactly the roles chosen	G, J
Project manager / task owner	Sets up projects, review policies and tasks	Configure review rules; load a dataset; choose the annotation surface; run the AI first pass; start, pause and complete tasks; reopen a finalised item; trust every figure; release data	B Setting up work; C AI; H Release
Annotator	Does first-pass labelling	Find available work; work without seeing colleagues' answers; never lose or have overwritten their own work	D Review; B
Reviewer	Accepts, corrects or rejects work	Judge a named person's submission; give reasons; escalate when unsure; never review their own work	D; C5
Expert (arbitrator)	Settles disputes	See escalated items; rule independently of the people involved; have the ruling recorded as its own decision	E Disputes
Quality lead	Assesses the dataset and the process	Repeatable runs, reviewer agreement, the AI's influence on people, and complete provenance	I Evaluation; F Provenance

1.4 From user needs to sprints

Table 3 traces each need through the story that captures it and the tickets that built it, split into back end (API and data) and front end (web), with the sprint each

landed in. Figure 1 shows the same work on the sprint calendar, one lane per user. A typical example: an administrator needs to bring people in (G5, J3); the broken invitation API was fixed first (SCRUM-8, #16, W7); the member-management screens were built on it the following sprint (SCRUM-90, #28, W8); and the client's rule that roles are chosen at invitation, with the invitee accepting on screen, followed in the break (SCRUM-115, #56). In general the API and data model land first and the screens follow, which is why many user needs span two sprints.

Table 3. User needs traced to stories, back-end and front-end tickets, sprints and pull requests

User	Need	Story	Back end: ticket (PR, sprint)	Front end: ticket (PR, sprint)
Administrator, invitee	Bring people in with the right roles	G5, J3	SCRUM-8 invitation flow restored (#16, W7)	SCRUM-90 member management (#28, W8); SCRUM-115 roles at invitation, invitee accepts (#56, break)
Administrator	Keep each project's work inside it	G1, G3	SCRUM-44 roles drive permissions (#4); SCRUM-21 cross-project write guards (#14, W7)	None needed (enforced by the server)
Task owner	Set how work is reviewed	B2	SCRUM-20 review policy (#25, W7); SCRUM-50 policy resolved into task rules (#37, W8)	Policy page (#25, #37); policy shown in the review panel
Task owner	Load a dataset; choose how items are annotated	B5, B7, B8	SCRUM-34 all-or-nothing registration (#11, W7); SCRUM-111 JSONL import (#42, break); SCRUM-112 annotation surfaces (#36, W8)	Upload with per-file field selection (#42); surface and AI model chosen at setup (#36); label definitions (#27, W7)
Task owner	Start, pause and close a task	B4, B6	SCRUM-43 one completion rule (#26, W7); SCRUM-24 lifecycle (#41, W8)	Activate, pause, resume and complete controls (#41, W8)
Task owner	AI first pass on a large batch	C1–C3	SCRUM-6/7 multi-provider client (#8, W6); SCRUM-1/2/3/46 job tables, worker, retry, trigger (#20–#22, #29, #30, W7)	SCRUM-30 AI run status (#7, W6); SCRUM-5 batch progress (#32, W8)
Annotator	Find work; work independently; never lose it	D5, D6, D8	SCRUM-25 draft ownership (#13, W6); SCRUM-28/26 locked items, atomic submit (#19, #23, W7); SCRUM-48 role-checked queues (#33, #35, W8)	SCRUM-114 work panel (#43, break); SCRUM-93 available-work list (#49, break)
Reviewer	Judge the right person's work, with reasons	C5, D2, D4	SCRUM-29 history protected (#6, W6); SCRUM-31 structured reasons (#17, W7); SCRUM-27 review names its submission (#34, W8)	Review form with reasons (#17); SCRUM-113 submission chooser (#44, break)
Reviewer	An independent second opinion	D7	SCRUM-51 sampled blind second review, agreement report (#53, break)	Blind second review in the panel (#53)
Expert	Settle disputes independently	E2, D1, E3/E4	SCRUM-35 send back (#17, W7); SCRUM-116 and SCRUM-52 independence guards (#45, #53, break); SCRUM-99/100 adjudication record (#48, paused)	Dispute desk (inherited, now on guarded routes); dispute screens E6 in W10
Project owner	Correct a finalised answer safely	D9	SCRUM-110 reopen with reason, old answer kept (#46, break)	Reopen action on a finalised item (#46)
Quality lead	Measure quality repeatably	I1–I5	SCRUM-68 harness (#39, break); I2–I5 in W9–W11	Measures reported from harness runs (W10)

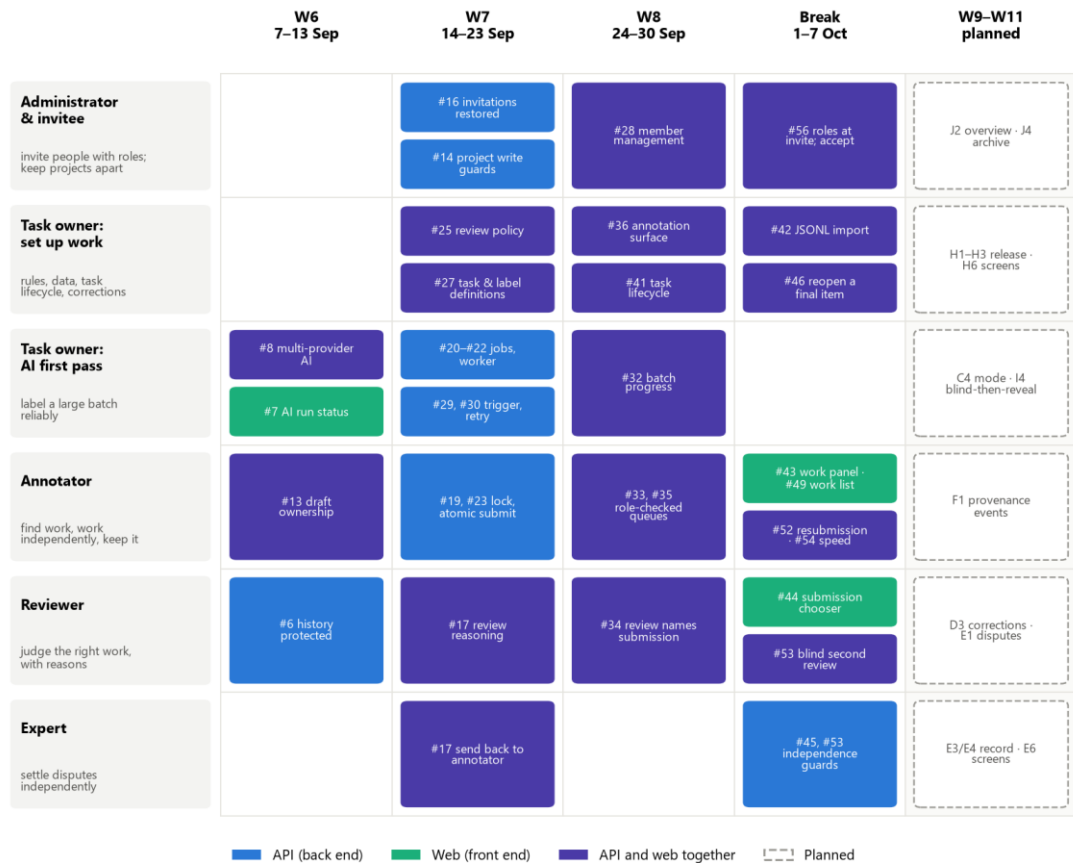


Figure 1. Merged pull requests by user and sprint (placed by merge date), with the work planned for Weeks 9–11

1.5 Sprint by sprint: what was built and what was solved

Table 4 reads the same work against our Jira sprints: each sprint's goal, its planned and completed load, what it built and the problems it solved. Many of those problems are defects from our audit of the inherited code (numbered as in Table 8); others were found while building or reviewing. The challenges met inside each sprint are in Section 2.2.

Table 4. Jira sprints since the proposal: goal, load, what was built and problems solved

Jira sprint	Planned / done	Built	Problems solved
W6 Governance & data-integrity 7–13 Sep	not pointed	Draft ownership (D6); legacy review writes retired (D2); multi-provider AI client and AI run status (C1, C3); roles (G1)	Anyone could edit another's draft (critical defect 6); the legacy API could rewrite approval history (critical 8); failed AI calls produced fake annotations (20); environment and reset script broken (23, 24)
W7 Lock down review data 14–23 Sep	20.5 / 19	Invitations, project write guards, finalised-item lock and atomic submission, PostgreSQL with two-database CI, AI job tables, worker, retry and trigger, review policy, task and label definitions, one completion rule	Finalised work silently overwritten (critical 2); submission not atomic (11); cross-project writes (9); invitations broken (17, 18); invalid statuses saved and a 409 reported as 500 (10, 29); dataset registration not transactional (16); no send-back (15); a 1,000-item AI batch held one request open for hours
W8 Route work, record decision 24–30 Sep	22 / 20.5	Role-checked work queues with a per-task submission limit; review names its submission; policy governs review; annotation surfaces; task lifecycle; member management; AI batch progress	Reviewer shown the wrong annotator's work (3); a task could never leave draft (28); a task with items could not be deleted and returned 500 (30); every non-empty queue returned 500 (found in review); reissuing an invitation could undo an acceptance
Mid-semester Break 1–7 Oct	19.5 / 15	Evaluation harness; answer versions; owner reopen; blind second review and adjudicator independence; work panel, submission chooser and available-work list; JSONL import; invitee acceptance	Self-adjudication on disputes (critical 7, fix merged); a reopened item flipped back by the next read; resubmission allowed on approved work; review decisions the queue would not offer; Save draft reverting a returned item's edits; a 200-item page sending 404 requests (2,969 ms); duplicated history entries (22)
W9 Trace decisions (current) 8–14 Oct	20 / in progress	Provenance events (F1), versioned guidelines (F2), corrections (D3), one authoritative answer (H4), release artefact (H1), automatic disputes (E1), mode on every item (C4), repeatable runs (I1)	To solve: reviewer corrections not saved (critical 5); conflicting exported answers (critical 4)
W10, W11 15–28 Oct	20.5, 7.5	Release gate and manifest, dispute screens, Expert Gate, item timeline, measures; real-data screens and integrity checks	Planned (Tables 16–18)

1.6 Front end: what each user can now do

This section describes, user by user, what the web application now lets each person do and what we changed to support it. The screenshots were taken on 8 October from the main branch running locally, using the client's AG News text dataset: one project with two tasks, one annotated by two people per item and one given an AI first pass by a local Llama 3 model. Names such as Dana Lee, Frank Ng and Bob Johnson are seeded test accounts.

1.6.1 Administrator and invitee

The members page (Figure 2) lists everyone in the organisation with their account status, membership status and roles, and lets the administrator create users, edit roles and status, and remove members (SCRUM-90, #28). When inviting, the administrator must choose the roles, because the client asked for no default role, and the invitee sees pending invitations in their own list and accepts one, receiving exactly those roles (SCRUM-115, #56). Two front-end faults found in review were fixed on

the way: reissuing an invitation could undo an acceptance, and switching organisation could let a late response overwrite the new organisation's data.

NAME	EMAIL	ACCOUNT STATUS	MEMBERSHIP STATUS	ROLES	JOINED	REMOVED	ACTIONS
Alice Chen	alice@example.com	active	active	admin	2026年10月8日 06:44	—	Edit roles Edit status Remove
Bob Johnson	bob@example.com	active	active	reviewer	2026年10月8日 06:44	—	Edit roles Edit status Remove
Test Annotator	annotator.test@example.com	active	active	annotator	2026年10月8日 06:44	—	Edit roles Edit status Remove
Test Reviewer Two	reviewer2.test@example.com	active	active	reviewer	2026年10月8日 06:44	—	Edit roles Edit status Remove

Figure 2. The administrator's members page: roles, status and the actions available on each member

1.6.2 Project manager and task owner

The policy page (Figure 3) shows the organisation's defaults and the project's own review policy, and marks which values are *Enforced*, meaning they already change what the workflow allows, and which are *Recorded*, stored and shown until the story that enforces them lands (for example the Expert Gate, G2, in Week 10) (#25, #37). Task setup now chooses the annotation surface, such as classification or text spans, and the AI model (#36); defines labels with their meanings (#27); and uploads JSONL, JSON, CSV or text files, choosing per file which fields become the item that annotators and the AI see (#42). The task header carries activate, pause, resume and complete controls (#41); the AI run shows batch progress and each item's job status (#7, #32); and a finalised item can be reopened by the project owner with a reason (#46).

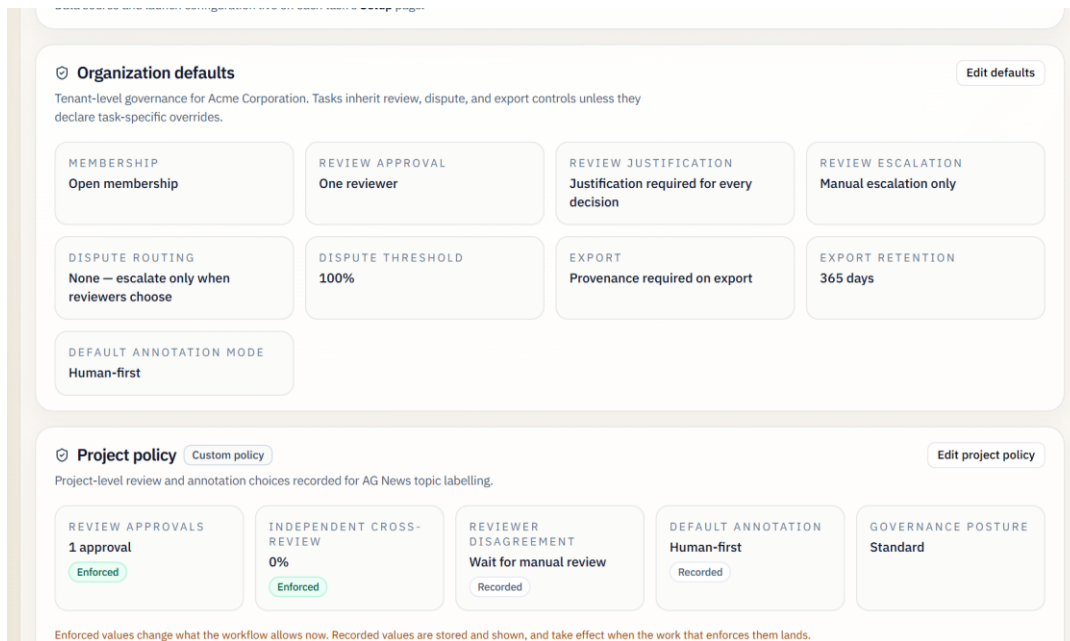


Figure 3. Organisation defaults and the project policy, distinguishing enforced from recorded rules

1.6.3 Annotator

The annotator's work list (Figure 4) shows only items their role allows and how many of the required human submissions each has. An item showing "2 of 2 submitted" has its Annotate button greyed out, while one showing "0 of 2" is open for work (#49). Opening an item brings up the work panel, which no longer shows colleagues' drafts or answers, keeps edits on returned work instead of reverting them, and reports any refusal in the server's own words (#43). The Annotate and Review pages now load in about half a second rather than three (#54).

Items

Human annotation happens item by item. Open any item into the work panel to produce the first-pass output.

Task item filters

6 visible

Q

Search task items or external refs

Draft

Returned

Rejected

ITEM	STATUS	PROGRESS	DRAFT OUTPUT	ACTIONS
ITEM_922B12AA279B ag_news_subset.jsonl#3	Rejected	2 of 2 submitted	unknown	Annotate
ITEM_B1CA7F55A23F ag_news_subset.jsonl#6	Submitted	2 of 2 submitted	unknown	Annotate
ITEM_3B0985D3A0CD ag_news_subset.jsonl#7	Submitted	2 of 2 submitted	unknown	Annotate
ITEM_9E56629BC924 ag_news_subset.jsonl#8	Not started	0 of 2	unknown	Annotate
ITEM_88F73EBA3BFD ag_news_subset.jsonl#9	Not started	0 of 2	unknown	Annotate
ITEM_B27BAE9D79CA ag_news_subset.jsonl#10	Not started	0 of 2	unknown	Annotate

Figure 4. An annotator's work list: items that already have their two submissions cannot be taken

1.6.4 Reviewer

The review panel (Figure 5) lists the submissions awaiting this reviewer on an item, by author, and the reviewer picks one; the decision is recorded against that submission only (#34, #44). The panel shows the task's review policy, which here requires one approval and a justification for every decision, and offers accept, adjust, reject and escalate. On an AI-assisted task the AI's answer arrives for review in the same panel, labelled as AI-assisted (Figure 6). A sampled share of submissions is shown to a second reviewer who cannot see the first decision until their own is in (#53).

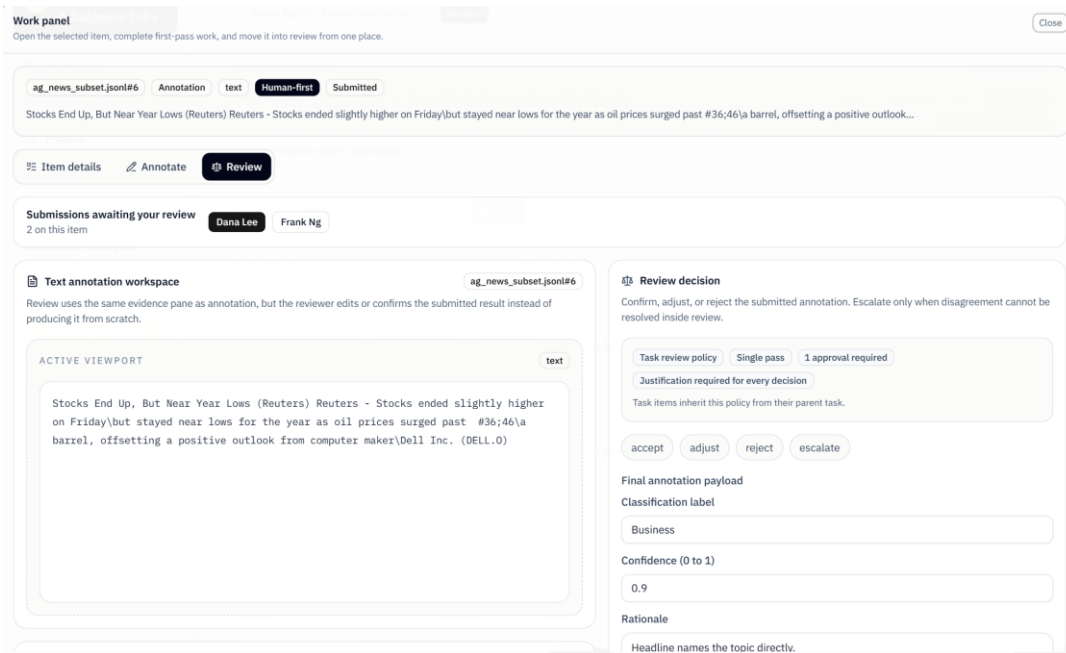


Figure 5. The review panel: two submissions await review on this item, and the reviewer picks one

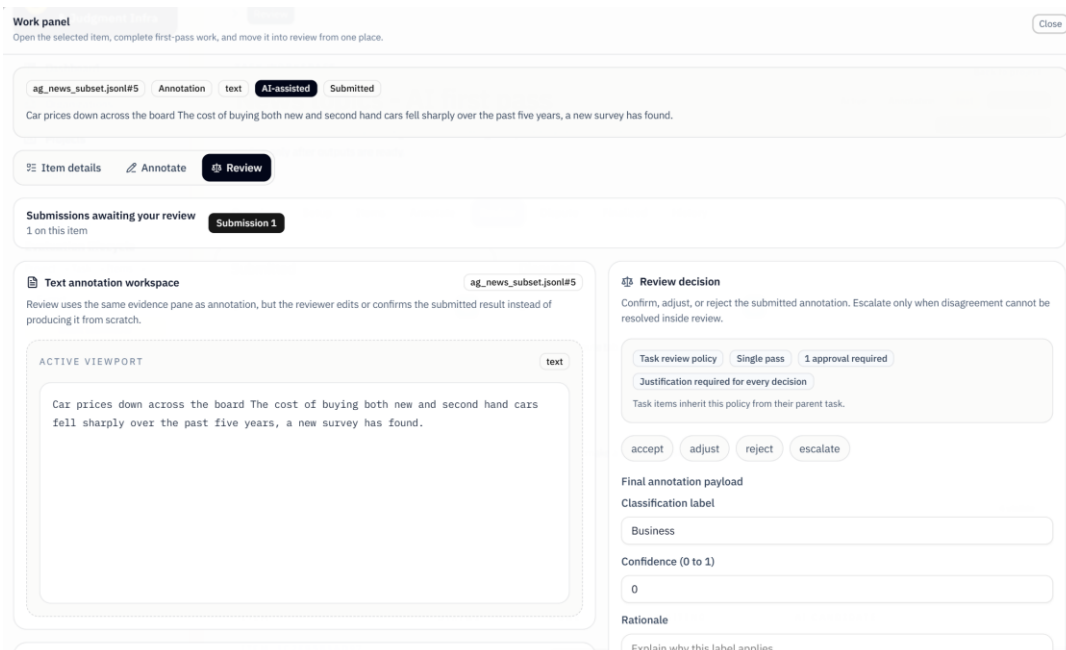


Figure 6. A reviewer judging the AI's first pass (Llama 3 labelled this headline "Business")

1.6.5 Expert

The dispute desk (Figure 7) lists items escalated to an expert, with the reviewer's reason, and records the expert's decision, either to finalise the item or to send it back to the annotator, together with a note. The server refuses the decision from anyone who annotated, reviewed or routed the item, administrators included (#45, #53). Once the expert rules, the item joins the task's finalised list (Figure 8) and can be exported. The

client's refined three-outcome ruling and the dedicated dispute screens are Week 9–10 work.

Figure 7. An escalated item waiting for an expert's ruling, with the reviewer's reason for escalating

Figure 8. Finalised items, including item #5 after the expert's ruling, ready for export

1.7 Back end and data

1.7.1 The dataset

No new data is collected. The client supplied about 4 GB in four modalities (Table 5), each a task-aligned subset: a *source* folder with the records or media files, and a *handoff* folder of JSONL files describing the task and its label list. Text arrives **structured**, as JSONL records that each hold an identifier, the text and gold labels, while images, audio and video are **unstructured** media files. Raw client data is not copied into the database: items point to it through data pointers, which keeps the client's data where it lives.

Table 5. The client's datasets and how each kind enters the platform

Modality	Datasets (task, size)	Form	How it enters Hej
Text	AG News (topic classification), FewNERD (named entities), SemEval-2010 Task 8 (relations), Civil Comments (moderation judgement); 200 items each	Structured: JSONL records with id, text and gold labels; JSON label lists	JSONL import (#42): one item per record; the task selects which fields annotators and the AI see, so gold labels stay hidden; the source file's hash is recorded on each item
Image	BoneBreak (classification), Retail Product Checkout (inventory judgement); 200 files each	Unstructured image files	Upload: one item per file, referenced by pointer
Audio	LibriSpeech (transcription), MInDS-14 (intent); 200 WAV files each	Unstructured audio	Upload; transcription through an external speech API (whisper-1)
Video	UCF101 (action recognition, 202 files), MOT17 and UOT32 (object tracking)	Unstructured video and tracking sequences	Upload; kept working but not extended, since text is the demonstration path

1.7.2 AI: external models, not trained by us

We do not train or fine-tune any model. The platform calls external large language models through one OpenAI-compatible client, which can reach hosted APIs from OpenAI, Qwen (DashScope), DeepSeek, Doubao (Ark) and Groq, or a model run locally through Ollama, as in this report's screenshots. It records which provider and model produced each answer. The AI's output is treated as one more author's answer that people must review, never as ground truth; a failed call is reported as a failure rather than turned into a placeholder answer (C1–C3).

1.7.3 How the back end supports the front end

Every screen described in Section 1.6 rests on back-end work, usually merged a sprint earlier. Table 6 pairs each back-end change with the front-end feature it enables and the users it serves.

Table 6. Back-end changes and the front-end features they enable

Back-end change (API, data model)	Front-end feature it enables	Users
Invitation lifecycle and role assignment (#16, #56)	Members page: invite with roles, pending invitations, accept (#28, #56)	Administrator, invitee
Capabilities from current roles; cross-project write guards (#4, #14)	Pages offer only what a role may do; refusals show the server's reason	Everyone
Policy resolver: project policy into task rules (#25, #37)	Policy page with enforced and recorded values; policy in the review panel	Task owner, reviewer
Text intake service: one item per record, field selection, source hash (#42)	Upload with per-file field selection	Task owner
Annotation-surface catalogue (#36)	Setup chooses the surface; the editor follows it	Task owner, annotator
Task lifecycle endpoints and guards (#41)	Activate, pause, resume and complete controls	Task owner
AI job tables, background worker, retry and dead-letter (#20–#22, #29, #30)	Batch progress and per-item job status (#32); AI answer in the review panel	Task owner, reviewer
Role-checked queues, submission limit, independence rules (#33, #35)	Available-work list with "2 of 2"; work panel without colleagues' answers (#43, #49)	Annotator, reviewer
A review names the submission it judges (#34)	Submission chooser (#44)	Reviewer
Cross-review sampling and blind second review (#53)	Second reviewer sees the first decision only after deciding	Reviewer, quality lead
Escalation routing and decision guards (#45, #53)	Dispute desk refuses non-independent experts	Expert
Answer versions with one authoritative answer (#47)	Finalised list and export carry every author's answer	Task owner
Reopen endpoint keeping the old answer (#46)	Reopen action on a finalised item	Project owner
One batched read of a task's drafts, new indexes (#54)	Annotate and Review pages load in ~0.56 s instead of ~3 s	Annotator, reviewer

1.7.4 Back-end scripts and services used in operations

Some back-end pieces are not screens but are run by the team or an operator to make the platform usable. Table 7 lists them with what each enables for users.

Table 7. Operational scripts and services, and what they enable

Script or service	What it does	What it enables, and for whom
init_data.py --reset	Rebuilds the database with two demonstration organisations, eight accounts covering every role, and sample projects, tasks and items	The team and the client can sign in as any role and walk the whole workflow; used for demonstrations and manual tests
AI batch worker (HEJ_AI_EXECUTION_MODE=worker)	A separate process that claims AI jobs, calls the model, retries transient failures with back-off and dead-letters permanent ones	Task owners start an AI run and watch its progress instead of waiting on one long request
Evaluation harness (python -m evaluation run)	Runs every casebook case through the real API as real users in a fresh database and writes a JSON report	Quality lead and developers check behaviour before merging; the basis of the Week 10–11 measures
Docker Compose with PostgreSQL 18	Starts the database, the API and, optionally, the AI worker	Every member and the client run the same production-like environment
Schema migration at start-up	Adds new columns to an existing database	Existing demonstration data survives upgrades
CI workflow	Runs the backend suite on SQLite and PostgreSQL on every push	Reviewers see whether a pull request breaks either database

1.7.5 Records, quality and inherited defects

Every answer is now kept as a version of its own, whether it comes from the AI, an annotator or a resubmission, and at most one version per item can be marked authoritative, which the database itself enforces; reopening an item clears the marker but keeps every version (F3; #47). This is what the release work in Weeks 9 and 10 builds on. Each action also writes an audit entry with its actor and time (Figure 9). The evaluation harness (#39) runs scripted cases through the real API as real users holding real roles, and its four cases pass; a draft PR (#51) adds the casebook's adversarial categories and gold records. The platform runs on PostgreSQL 18 as well as SQLite, and CI runs the backend suite on both, with PostgreSQL set to a non-UTC time zone to expose date-handling bugs (#24).

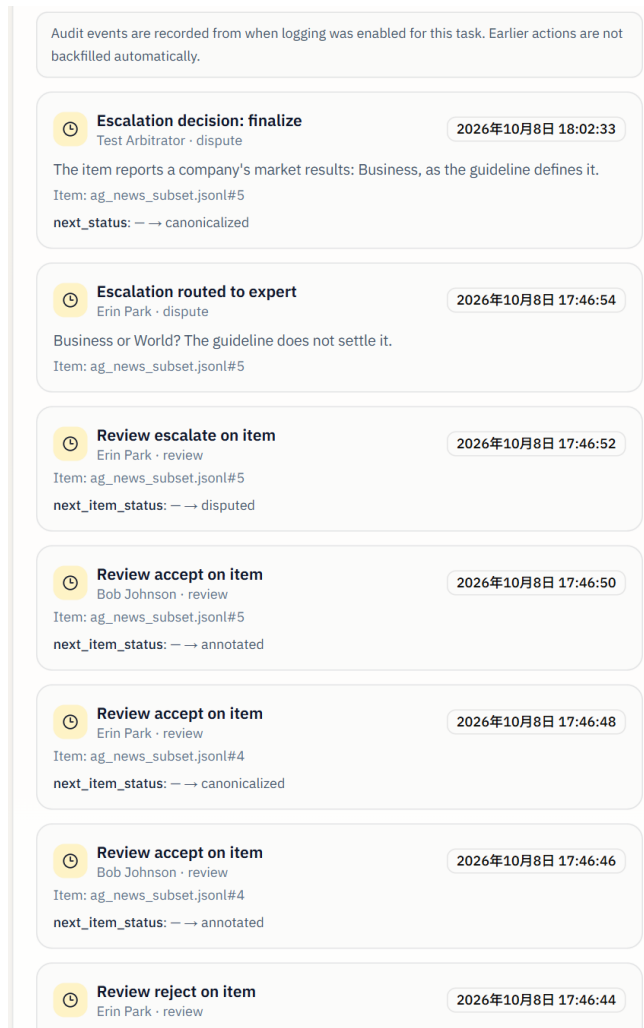


Figure 9. A task's activity history, newest first: the expert's ruling, the routing and the reviews, each with its actor and time

Our audit of the inherited code catalogued 30 defects. Table 8 lists the six rated critical, which were the ones that made the platform unsafe for real multi-user work, and where each stands. Of the other 24, 17 are fixed, 5 have a fix merged that awaits a closure check, and 2 are open: completion and export figures disagreeing (issue 19, story B6) and the wrong actor type in audit entries (issue 21, F4 in Week 10).

Table 8. The six critical inherited defects

#	Defect	Status	How it was or will be closed
2	A finalised annotation could be silently overwritten	Fixed (W7)	D5: finalised items refuse draft create, update and submit (#19)
6	Drafts had no ownership, so anyone could edit another person's draft	Fixed (W6)	D6: an unfinished draft belongs to its author (#13)
8	The legacy review API could rewrite approval history	Fixed (W6)	D2: the four legacy write routes return 410 Gone (#6)
7	Annotators could approve their own work	Fix merged; check pending	Approval and both dispute paths guarded (#45, #53); closure check in W9
5	Reviewer corrections were not saved	In review	D3: a correction is stored whole beside the answer (PR #58, Jingwei)
4	A finalised item could export conflicting answers	Open, W9	H4: at most one authoritative answer per item (Dishank)

1.8 Deliverables and the MVP

Table 9 assesses each deliverable from the proposal against its own definition of done, with the weeks it was planned for and the weeks it is actually delivered in, so each outcome can be read against the schedule in Section 3. Deliverables 1 and 7 are complete to date; 2 and 3 are largely built; 4 and 5 are the focus of Weeks 9 to 11.

Table 9. Status of the seven proposal deliverables on 8 October

Deliverable (planned → actual weeks)	Status	Evidence and what remains
1 Engineering foundations (W5–W6 → W5–W7)	Achieved	Platform builds and runs from a clean clone; test suites restored and made truthful (PR #2); hard-coded credential removed; every change merged through a reviewed PR; CI runs the backend suite on SQLite and PostgreSQL on every push (#24, W7). Gap: GitHub branch protection is not switched on, so review is a team rule rather than enforced.
2 Core end-to-end workflow for text (W7–W10 → W6–W10)	MVP achieved	Project → policy → task → JSONL import → AI first pass → human annotation → review → escalation → expert ruling → export runs through the user interface on the client's AG News data (Section 1.5). Remaining: automatic disputes (E1, W9), the client's refined adjudication record (E3/E4, paused on issue #40) and the dispute screens (E6, W10).
3 Provenance and governance (W9–W10 → W6–W10)	Governance achieved; provenance in progress	Roles, invitations, project boundaries, self-review guards and independent second review are merged. Every answer is now its own version with at most one authoritative (F3). Remaining: the provenance event record (F1) and versioned guidelines (F2) in W9; item timeline and actor kinds (F4, F5) in W10.
4 Release validation and export (W10–W11 → W9–W11)	Not started (on plan)	Export of finalised items works and follows one completion rule (B6). Week 9: one authoritative answer per item (H4) and the immutable release artefact (H1). Week 10: the release gate (H3) and manifest (H2). Week 11: release, policy and history screens on real data (H6).
5 Evaluation harness and casebook (W10–W11 → break–W11)	Ahead of plan	Harness merged (#39); its four end-to-end cases pass. Casebook categories and adversarial cases in a draft PR (#51). Repeatable runs in W9; 40 cases and agreement/AI-influence measures in W10; integrity checks and findings record in W11.
6 Documentation and handover (W12 → 29 Oct – 1 Nov)	In progress	Architecture decision records are written as each client decision lands; the technical report, per-member decision records and demonstration are scheduled for 29 October – 1 November (Table 18).
7 Assessed artefacts (W5, W9, W13 → unchanged)	On schedule	Proposal submitted (W5); Status Checks 1 and 2 held with the client on 26 August and 23 September; this progress report (W9); Status Check 3 with demonstration (W12); final presentation and reports (W13).

1.8.1 The MVP: complete, tested and working

At Status Check 1 (26 August) the client and the team agreed that success is first defined as a working, high-quality end-to-end annotation workflow for text, covering task setup, AI annotation, human review, role-based permissions and dispute resolution, with further functionality treated as extension work. That is our minimum viable product. Table 10 walks through it step by step, with the screen that shows each step working and the automated tests that check it; every test module named is on main, and every one passed when the suites were re-run for this report on 8 October.

Table 10. The MVP journey: where each step is shown working and how it is tested

Step	Who	Shown working	Tested by (backend modules; harness cases)
1 Invite members and choose their roles	Administrator	Figure 2	test_invitations, test_admin_iam_routes, test_role_separation_routes
2 Create a project and its review policy	Administrator, task owner	Figure 3	test_project_policy_enforcement, test_policy_resolver, test_governed_action_routes
3 Create a task, import a JSONL dataset, activate it	Task owner	Section 1.6.2	test_text_intake_service, test_text_intake_routes, test_task_lifecycle
4 AI first pass on every item	Platform (AI model)	Figure 6	test_ai_batch_worker, test_ai_preannotator, test_ai_run_progress; EV-002
5 Independent human annotation, up to the task's limit	Annotators	Figure 4	test_annotation_independence_privacy, test_task_required_annotators, test_draft_submission_atomicity
6 Review a named submission, with a justification	Reviewer	Figure 5	test_review_annotation_selection, test_review_justification, test_independent_second_review; EV-000, EV-003
7 Escalate; an independent expert rules	Reviewer, expert	Figure 7, Figure 9	test_adjudication_independence, test_review_actions; EV-001
8 Finalised item is locked and exported	Task owner	Figure 8	test_finalised_item_writes, test_task_exports, test_export_provenance

Verdict. The MVP is complete: all eight steps run through the user interface on the client's data, and the backend (1,283 tests), frontend (400 tests) and end-to-end harness (4 cases) all pass. Two refinements the client asked for after the MVP was agreed are still open and are planned, not missing: the expert's ruling currently finalises an item or sends it back, and the client's three-outcome version (accept, return or reject) waits on issue #40; export currently delivers finalised items, and the gated release with a manifest is Deliverable 4 (Weeks 9–11). Provenance, release and evaluation extend the MVP to the full brief, and are where the remaining weeks go.

1.9 Progress by epic

Figure 10 shows the 63 stories by epic. Setting up work (B) is complete and review (D) and governance (G) are mostly complete; provenance (F), release (H) and evaluation (I) hold most of the remaining work, which is why they dominate the plan

in Section 4. Four stories were added after the proposal's 59: B7, B8 and D9 from the client's answers of 24 September, and A6 from the performance investigation. Six stories were set aside by team decision on 14 September: A1 (enforced branch protection) and A5 (one shared status vocabulary) as lower value than the remaining features, and K1–K4 (technical report, decision records, findings and demonstration), which are deliverables rather than features. They are tracked outside Jira and scheduled for 29 October to 1 November, not dropped.

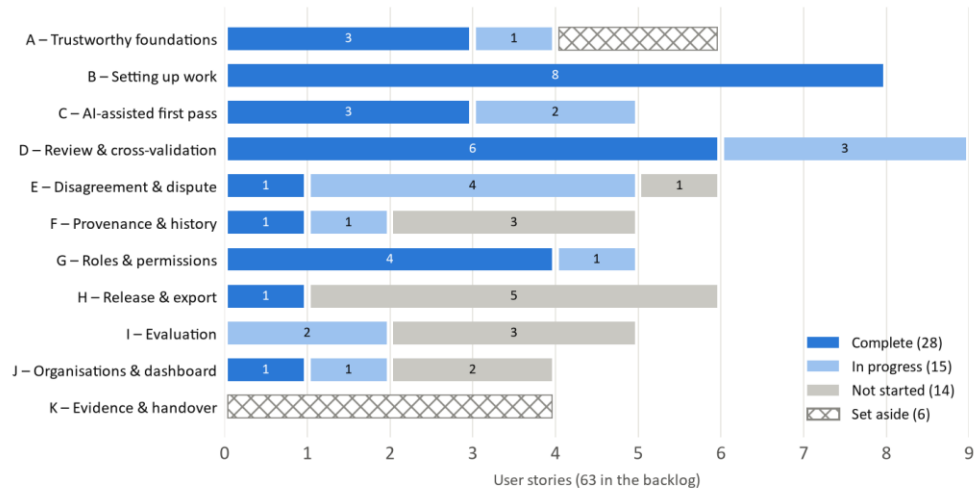


Figure 10. User stories by epic and status, from the team's records on 7 October

1.10 How we work

Several practices produced these results and will carry the project to the end:

- **Weekly sprints aligned with the client.** Sprints run Thursday to Wednesday so each closes after the Wednesday client meeting; at each weekly meeting finished tickets are closed, carry-over is planned first, and new work is packed into groups of 2–3u within each person's 1.5–3.5u.
- **Three records kept in step.** The Jira board (tickets and sprints), the contribution tracker (every PR, its reviewer and the review outcome) and a client-facing backlog page are reconciled weekly, and a story is reported complete only when its code is verified on main.
- **Specification first.** Shared mission, technology and roadmap documents; a feature specification per story (requirements, plan, validation checks); and an architecture decision record for each client decision.

- **Client decisions in writing.** A question log with a *needed-by* date and the stories each question blocks; two written rounds of answers; since 3 October, questions go to the client as GitHub issues.
- **Review as a habit.** Every PR is reviewed by another member, often with written review notes, and multi-user behaviour is checked by manual walkthroughs in two browsers.

2. OBSTACLES

This section separates the obstacles we have already met, both project-wide (Section 2.1) and inside particular sprints and stories (Section 2.2), which explain most of the timeline changes in Section 3, from the risks (Section 2.3) and dependencies (Section 2.4) still ahead, which shape the plan in Section 4; Section 2.5 records what we have learned. Each obstacle and risk carries an identifier (O1 to O9 and R1 to R10) and names the deliverable and the part of the timeline it touches, so it can be followed into Sections 3 and 4.

2.1 Obstacles met so far

Table 11. Obstacles met, their effect, our response and where each stands

ID	Obstacle	Effect (deliverable; timeline)	What we did	Status
O1	Inherited code unsafe for multi-user work: 30 defects, 6 critical (overwritten finalised work, editing others' drafts, self-approval, conflicting exported answers); failing and vacuous tests; a live API key in source	Almost every feature touched defective code; a separate clean-up phase would have used Weeks 6–7 (Del. 1, 2; Table 15 row 2)	Agreed with the client (26 Aug) to attach each defect to the story whose behaviour it affects; fixed only the defects that blocked building and testing first; each fix ships a test that fails on the old code	20 fixed, 6 awaiting closure check, 4 open and scheduled
O2	No review discipline in the inherited repository; changes had been overwritten	Regression risk with eight people working in parallel (Del. 1)	Every change through a PR reviewed by another member; CI on both databases from Week 7	49 PRs merged with review; branch protection still not enforced (A1 set aside)
O3	Requirements clarified only as features were built: per-author answer versions instead of one canonical answer, and no assignment of work to people (17 Sep); policy changes as new versions (21 Sep); expert adjudication made final (24 Sep)	Tickets rescoped mid-work (the queue API on the day it started); four stories added (B7, B8, D9, A6); D3, F3 and H4 blocked until 24 Sep (Del. 2–4; Table 15 rows 1, 5)	Question log with deadlines and blocked stories; two written Q&A rounds; every blocked roadmap group names a fallback; decisions recorded as ADRs and tickets updated the same day	Every question that blocked P0 work answered by 24 Sep
O4	Two product questions open: does a dispute concern the whole item or one answer, and may an expert merge answers (issue #40); who reworks an AI first pass a reviewer returns (#38)	E3/E4 (PR #48) paused since 3 Oct; part of H4 waits; returned AI items are stranded on main (Del. 2, 4; Table 15 row 7)	Built the parts that do not depend on the answer (separate adjudication record, independence check, row lock); posted both options neutrally on #40; raised at the 7 Oct client meeting	Waiting on the client; contingency in R2
O5	Infrastructure change mid-sprint: moving to PostgreSQL meant ten test files hard-coded to SQLite, no migration tool for schema changes, and a period when GitHub Actions would not run because of an account spending limit	One member's Week 7 went to infrastructure (3u); everyone updated local settings on merge day (Del. 1; Table 15 row 3)	One shared test fixture; CI on both databases; one agreed migration order per week and reset development databases; full suite run locally as evidence while CI was down	Resolved; procedure continues (R5)
O6	Parallel work on the same code: three tickets adding refusals to draft submission; four tickets changing the review path in the break	Merge conflicts, and refusals in the wrong order give users the wrong reason (Del. 2)	Merge order agreed on the first day of each week; stacked PRs (#43 → #44); paired authors review each other's PRs; refusal order documented with the API (#35)	Managed weekly (R10)
O7	Sequencing: at Status Check 2 (23 Sep) provenance and evaluation, both heavily weighted deliverables, had not started, and the release export (W10) depended on a provenance record scheduled for W11	Release gate could not be finished; evaluation would be late and thin (Del. 3–5; Table 15 rows 5, 8, 9)	Turned the mid-semester break into a sprint for the evaluation harness and scenario testing; moved the provenance record from W11 to W9 (replan of 1 Oct); pulled answer versions (F3) into the break	Harness merged; F1 under way in W9 (R1)
O8	Records drifting from the code: a ticket marked Done although its interface existed on no branch; D2 marked complete with two subtasks unbuilt	Progress over-reported to the client (Del. 7, status reporting)	Roadmap status follows code verified on main; a tracker row counts only after its review passes; differences raised at the weekly sync	Ongoing control (R6)
O9	Performance found late: a 200-item task page made 404 requests (2,969 ms), and every read repaired data with a write, which also caused a workflow bug	Dataset-sized tasks unusable for the demonstration (Del. 2, 7 demo; new story A6)	Batched read (#54) and bug fix (#50); "reads never write" scheduled for W10 (SCRUM-121)	6–8 requests, about 560 ms

In the effect column, *Del.* numbers refer to the proposal deliverables in Table 9, and Table 15 is the deviation table in Section 3.2.

2.2 Challenges by sprint and story

The obstacles above are project-wide. Table 12 goes one level down: the concrete problems met while building particular stories, in the sprint they arose, and how each was solved. Most were caught in code review or scenario testing before they reached main, which is the review practice of obstacle O2 at work; the last column links each to the project-level obstacle or risk it belongs to.

Table 12. Technical challenges met while building, by sprint and story

Sprint	Story, ticket (owner)	Challenge	How it was solved	Link
W6	D6, SCRUM-25 (Hanchen)	AI-created drafts had no owner, so "only the author may edit" had to keep unclaimed AI drafts usable without opening a back door	Ownership guard with a required viewer; unclaimed AI drafts can be claimed, never edited by others (#13)	O1
W7	Infrastructure, SCRUM-94 (Parth)	Ten test files hard-coded SQLite, so a PostgreSQL CI job would have tested nothing; the SQLite settings a second writer needs (WAL, busy timeout) were missing; docker compose up hung	One shared test fixture taking the database from configuration; the settings added with tests; start-up fails fast and names what is missing (#24)	O5
W7	C2, SCRUM-2 (Michael)	A worker whose lease had expired could still write and overwrite another worker's result; a permanent configuration error took about 15 minutes to surface; compose ran the worker on SQLite while the API used PostgreSQL	Lease ownership checked on every write, retry limit enforced, permanent errors dead-lettered at once, worker moved onto the shared fixture (#22, #30)	O5
W7–W8	D8, SCRUM-48 (Kanishka; fixes Hanchen)	Rescoped the day it started, when the client ruled out assigning work to people; independence had to become part of the queue query; every non-empty queue returned 500 because route tests only checked refusals; CI would not start (spending limit)	Per-submission review queue with independence in the query; route tests with real roles; full suite run locally as evidence (#33, #35)	O3, O5
W8	J3, SCRUM-90 (Tim)	Reissuing an invitation could undo an acceptance; a late response after switching organisation overwrote the page	Renewal only while an invitation is pending; stale responses ignored (#28, second review round)	O6
W8	B4, SCRUM-24 (Dishank)	The web app could not activate a task, so nothing created in the interface could be worked on; queues still offered work on draft and paused tasks	Lifecycle controls added to the web app; queues respect task state; the AI first pass moved to activation (#41)	O3
W8	B7, SCRUM-111 (Yi)	Each item carried an unauthenticated link to the whole source file, gold labels included	Source kept private; items carry only the selected fields and the source hash; decision recorded in an ADR (#42)	O1
W8	C2, SCRUM-46 (Michael)	A dead-lettered item could never be re-run; the AI's annotation had no author	Re-run path added; provider and model recorded on the annotation; an AI author identity moved to F3 (#29, #47)	O3
Break	D7, SCRUM-51/52 (Parth)	Blindness never ended: a reviewer stayed blind to decisions after the round closed, and an adjudicator who could review was blind to the dispute; the agreement report leaked an open round's verdicts	Raised in review and addressed before the PR was approved by three reviewers and merged; one shared rule now decides both the adjudicator queue and the decision (#53)	O6
Break	D9, SCRUM-110/120 (Jingwei)	A reopen reason over 2,000 characters was a 500 on PostgreSQL; a reopened item was flipped back to expert send-back by the next page read	Length validated; status resolution respects the reopen (#46, #50); reads that write moved out in W10 (SCRUM-121)	O9

Sprint	Story, ticket (owner)	Challenge	How it was solved	Link
Break	A6, SCRUM-119 (Hanchen)	A 200-item Annotate page sent 404 requests and ran 6,882 SQL statements (2,969 ms); no single statement was slow	One batched read of items and drafts, with indexes: 6–8 requests, about 560 ms (#54)	O9
Break	E3/E4, SCRUM-99/100 (Yi)	Whether a dispute concerns one answer or the whole item was never settled with the client	Paused; the parts that do not depend on the answer are built (#48, draft); question on issue #40	O4, R2

2.3 Risks ahead

Table 13 is the risk register for the remaining weeks, rated for likelihood (L) and impact (I) as high, medium or low, and plotted in Figure 11. Each risk names what it affects, has more than one mitigation so that the plan does not rest on a single remedy, and has an owner and a trigger, the observable event that starts the contingency; both are reviewed at the weekly meeting. The dependencies behind R1 and R2 are drawn in Figure 14 (Section 4.2).

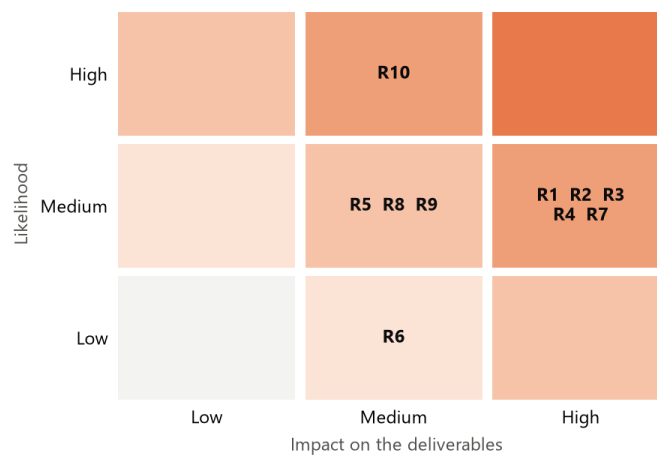


Figure 11. Risk heat map: the five medium-likelihood, high-impact risks drive the contingency plan

Table 13. Risk register for Weeks 9–13

ID	Risk and what it affects	L	I	Mitigations	Owner; trigger
R1	The provenance event record (F1) slips. The release gate (H3), manifest (H2), item timeline (F5), real-data screens (H6) and integrity checks (I5) all read its events. Affects Del. 3, 4; W10–W11	M	H	(a) F1's owner carries no other counted work in W9. (b) The event format is fixed early in the week so the W10 release work can build against it. (c) If F1 is late, W10 builds the gate's checks that need no events first (unreviewed, superseded and disputed items) and adds the provenance check when F1 merges. (d) W11's spare capacity absorbs the remainder	Parth; F1 not merged when W9 closes (14 Oct)
R2	Client answers on #40 and #38 arrive late or change scope. E1 → E6 dispute screens (W10) → E5 (W11), and part of H4, depend on them. Affects Del. 2, 4; W10–W11	M	H	(a) Keep building the scope-independent parts. (b) Keep the two options' differences inside one service. (c) If there is no answer by the Week 10 planning meeting, put the team's preferred option to the client in writing and build it, recorded as a reversible ADR. (d) H4 first covers the agreed cases: one authoritative answer per item, none for an ambiguous item	Hanchen; no answer by the W10 planning meeting

ID	Risk and what it affects	L	I	Mitigations	Owner; trigger
R3	No room after Week 11: build ends 28 Oct and no sprint follows. Affects Del. 2–5; W11	M	H	(a) W11 planned at 7.5u against a capacity of 12–28u. (b) P0 work first (H6). (c) J1 is the first story dropped, then P1 scope by agreement with the client. (d) Carry-over is planned before any new work	Whole team; W11 load above 12u after W10 carry-over
R4	Evaluation volume and quality: 40 cases and agreement and AI-influence measures by W10, findings by W11. Affects Del. 5; W10–W11	M	H	(a) Every story adds cases for its own behaviour. (b) The harness already drives the real API. (c) Scripted AI answers make runs repeatable. (d) Agreement reported with an established chance-corrected coefficient, never a single unexplained score	Hanchen, Tim, Parth, Michael; casebook below 40 cases when W10 closes (21 Oct)
R5	Several schema changes in W9 (F1, F2, C4) with no migration tool. Affects Del. 3; W9	M	M	(a) One migration order agreed on day one. (b) Development databases reset afterwards. (c) CI on both databases catches divergence	Parth, Michael, Jingwei; a migration conflict in CI
R6	Records overstate progress to the client. Affects Del. 7; every sprint	L	M	(a) Status from code verified on main. (b) Tracker review gate. (c) Weekly reconciliation of board, tracker and code	Hanchen; board or tracker differs from main at the weekly sync
R7	Handover deliverables (technical report, per-member decision records, findings, demonstration) are not on the Jira board. Affects Del. 6, 7; 29 Oct – W13	M	H	(a) 29 Oct – 1 Nov reserved for them, with owners (Table 18). (b) ADRs written with each decision feed the decision records. (c) I5's findings record feeds the findings. (d) The demonstration is rehearsed on the real screens once H6 lands, on this report's AG News set-up	Hanchen; no report outline by 28 Oct
R8	The client requires project-level isolation for reads as well as writes; writes are guarded, reads are not yet. Affects Del. 3; W11	M	M	(a) Raised as a product decision. (b) Built in W11's spare capacity if possible. (c) Otherwise recorded as a known limitation in the technical report	Hanchen; client confirms it is needed for the demo
R9	AI providers are non-deterministic or unavailable during evaluation or the demonstration. Affects Del. 5, 7	M	M	(a) The harness uses scripted AI answers. (b) The demonstration can run on a local Ollama model, as in this report. (c) Failures are shown, never disguised	Michael; a harness or demo run fails on a provider error
R10	Integration conflicts on the review path: three open PRs (#48, #58, #60) change the same functions. Affects Del. 2; W9	H	M	(a) Agreed merge order. (b) Whichever merges second rebases. (c) Authors review each other's PRs	Jingwei, Parth, Yi; a PR needs a rebase after another merges

2.4 Dependencies

Table 14 lists what the remaining work depends on, inside and outside the team, when each is needed and what happens if it is late. External dependencies are the ones we cannot schedule ourselves, so each has a fallback that does not need the outside party.

Table 14. External and internal dependencies

Dependency	Type	Needed by	Status (8 Oct)	If it is late
Client answer on dispute scope (issue #40)	External	E3/E4 now; E6 in W10	Open since 3 Oct	R2: written proposal and a reversible ADR
Client answer on reworking a returned AI first pass (#38)	External	AI-assisted tasks; the demo	Open since 3 Oct	Demonstrate rework on human-first tasks; record the gap as a known limitation
Weekly client meeting	External	Each sprint review	Held every Wednesday	Questions continue asynchronously on GitHub
Client datasets with gold labels	External	I2–I4 measures (W10)	Available locally	Not needed
AI providers (hosted keys, local Ollama)	External	AI runs; I4 (W10); the demo	Local Ollama verified on 8 Oct	R9: scripted AI answers

Dependency	Type	Needed by	Status (8 Oct)	If it is late
GitHub Actions CI	External	Every PR	Running	Full suite run locally and attached to the PR
F1 provenance events	Internal	H3, H2, F5 (W10); I5 (W11)	In progress (Parth)	R1 contingency
H4 authoritative answer; H1 artefact	Internal	H3, H2 (W10); H6 (W11)	Not started (W9)	Zero float: owners report mid-week at the client meeting
D7 disagreement comparison	Internal	E1 (W9)	Done (#53)	Not needed
I1 repeatable runs	Internal	I3, I4 (W10)	In progress (Hanchen)	I3/I4 start on single runs, add comparison later

2.5 Lessons learned

Looking back over Weeks 4 to 9, five lessons change how we run the rest of the project:

- **Schedule by dependency, not by build order (O7).** We built the platform first because it was the visible work, and only a dependency check at Status Check 2 showed that the release gate needed a provenance record planned after it. Since then every replan is checked against the dependency rules in our roadmap, summarised in Section 4.2, before it is accepted.
- **Ask before the ticket starts (O3).** A question answered after work begins costs a rescope; one asked with a needed-by date costs nothing. We now raise a question when the story is planned, with the date it is needed by, rather than when it is being built.
- **Treat the record as a claim to verify (O8).** A board that says Done is not evidence that code exists; status reported to the client now follows code verified on main.
- **Test with realistic volume early (O9).** Every test used a handful of items, so a page that sent 404 requests for 200 items looked fine until scenario testing. A6 now sets a measured target of under one second and at most ten requests for a 200-item page, which later changes are checked against.
- **A social rule is weaker than a technical one (O2, O6).** Review before merge has held because the team keeps it, not because GitHub enforces it; agreeing merge order on day one has kept parallel changes to the same code manageable, which branch protection alone would not.

3. DEVIATION TO TIMELINE

3.1 The baseline

The baseline is the Gantt chart in our proposal (Figure 12), refined at Status Check 1 (26 August) into an indicative weekly plan: Week 5, user stories and the review workflow; Week 6, schema and contract validation; Weeks 7–8, the core annotation backend; Weeks 9–10, human review, roles and disputes; Week 11, an end-to-end run on the provided datasets; Week 12, hardening, documentation and the demonstration; Week 13, the final presentation and report. The client accepted it as indicative, with changes raised at the weekly meeting.

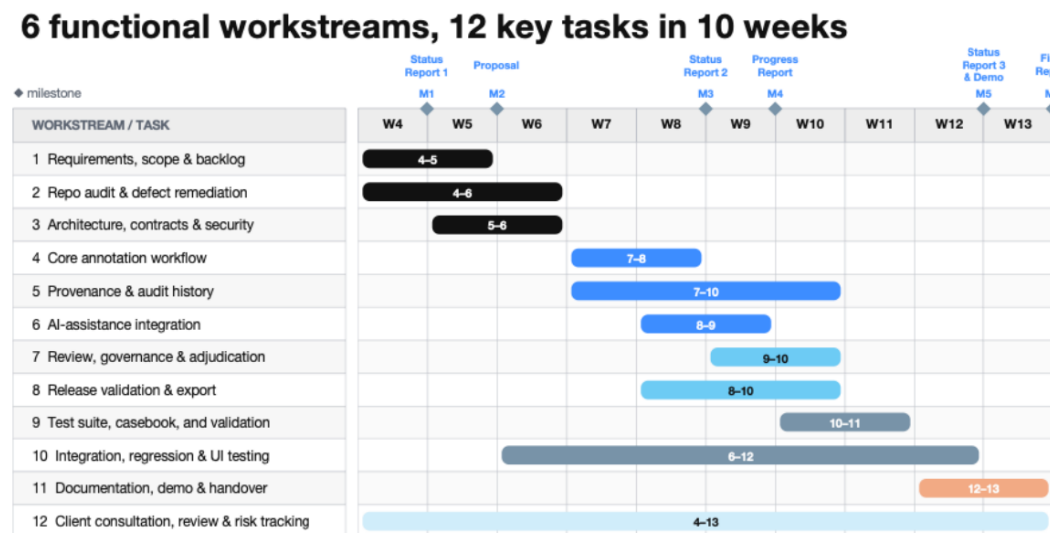


Figure 12. The original plan, reproduced from our Week 5 proposal

Calendar note. Since 19 September our sprints run Thursday to Wednesday so that each closes after the Wednesday client meeting. Sprint W9 therefore runs 8–14 October while teaching Week 9 began on 5 October; Week 7 absorbed the change and ran ten days, and the mid-semester break has its own sprint (1–7 October), which the proposal's chart did not include.

3.2 What moved, and why

Figure 13 overlays the proposal's plan (grey) on what has been delivered (solid) and what is re-planned (hatched). Table 15 explains each deviation and links it to the obstacle that caused it (Section 2.1) or the risk it creates (Section 2.2).

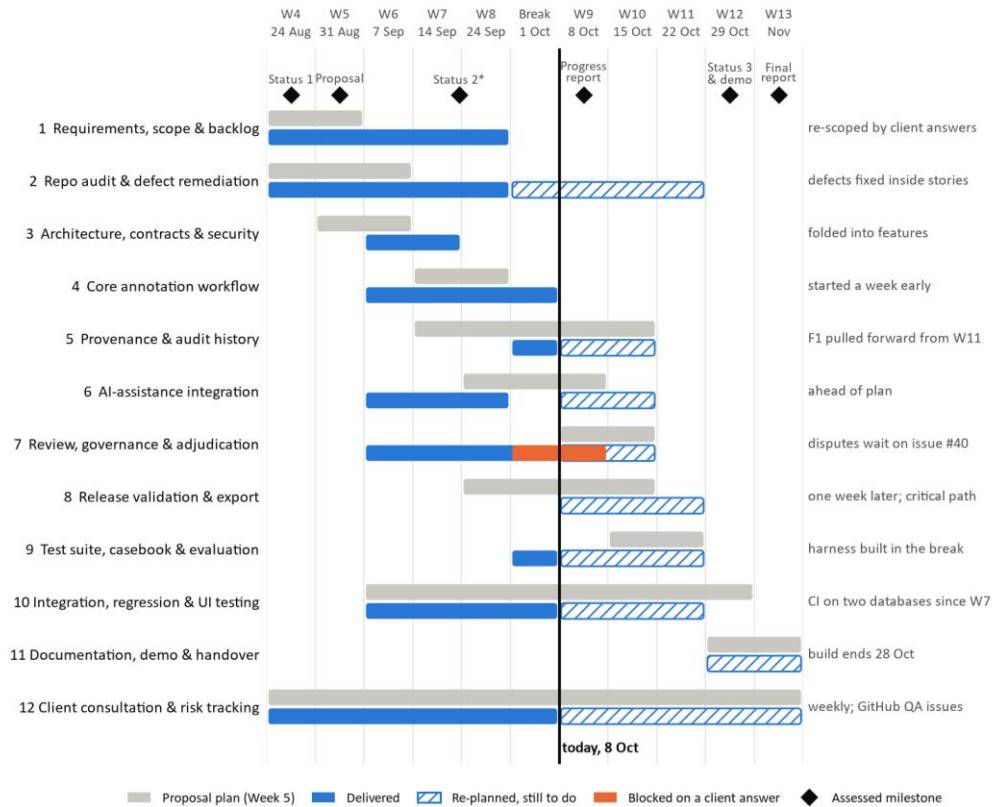


Figure 13. Planned versus actual timeline by workstream, as of 8 October

Table 15. Deviations from the proposal timeline

Workstream	Proposal	Actual or re-planned	Reason	Impact
1 Requirements and backlog	W4–W5	W4–W8; 59 → 63 stories	Client answers clarified scope as features were built (O3)	Four stories absorbed in W8 and the break; nothing dropped
2 Defect remediation	W4–W6, as a phase	Inside each story, W6–W11	Client direction of 26 Aug (O1)	Feature work began a week earlier; 4 defects remain, scheduled
3 Architecture and contracts	W5–W6 review	Separate review replaced by fixing mismatches in each feature; PostgreSQL migration added in W7	A standalone review was low value; a production database was needed for concurrent work (O5)	3u of W7 went to infrastructure
4 Core annotation workflow	W7–W8	W6 – break (done)	Dependencies resolved early	Ahead; base for review work
5 Provenance and audit history	W7–W10	F3 in the break; F1 moved W7 → W8 → W11 (24 Sep) → W9 (1 Oct); F2 W9; F4–F5 W9–W10	Capacity went to queues and client-driven stories; the release gate needs F1 first (O7)	Critical path; W9 is the heaviest week (R1)
6 AI assistance	W8–W9	W6–W8 (done); mode on every item W9; blind-then-reveal W10	Built on an existing multi-provider client; blind-then-reveal redefined by the client as an evaluation protocol (21 Sep)	Ahead
7 Review, governance, adjudication	W9–W10	From W6; most done by the break; disputes paused 3 Oct; E1 → W9, E6 W10, E5 W11	Governance defects were critical (O1); dispute scope still open (O4)	Dispute epic ends in W11, not W10 (R2)
8 Release validation and export	W8–W10	W9–W11	Depends on F1, F3, H4 and the client's release rule (answered 24 Sep)	One week later; H6 lands in the last sprint (R3)

Workstream	Proposal	Actual or re-planned	Reason	Impact
9 Test suite and casebook	W10–W11	Harness in the break; repeatable runs W9; 40 cases and measures W10; integrity checks W11	Status Check 2 found evaluation at zero (O7)	Earlier than planned; reduces R4
10 Integration and regression testing	W6–W12	W6–W11; CI on two databases from W7; scenario testing in the break	Build ends 28 Oct	Regression testing ends with the build
11 Documentation, demo, handover	W12–W13	29 Oct – 1 Nov, then W12–W13 assessments	Week 11 made the last sprint (1 Oct)	No build buffer in W12 (R3, R7)
Sprint calendar	Mon–Sun weeks, no break work	Thu–Wed sprints; the break became a sprint	Close each sprint after the client meeting; use the break	About one extra week of capacity

3.3 Carry-over between sprints

A ticket still in review at the end of a sprint counts as that sprint's work, and unfinished work moves into the next sprint before any new work is pulled in. From Week 7 to Week 8 we carried the work-queue API, the member-management screens, the project-policy settings and the self-approval check for disputes; from Week 8 into the break, the independent second review (SCRUM-51), review and state consistency (SCRUM-109), the available-work list (SCRUM-93) and invitee acceptance (SCRUM-115). All of these had merged by 7 October except SCRUM-109, now PR #60. Across sprints the team completed 77–93% of the planned load (Figure 15, Section 4.4), the shortfall being work in review when the sprint closed rather than work abandoned.

3.4 Impact on the project

- **End date and scope unchanged.** Every proposal deliverable remains; build work ends on 28 October as the Week 11 boundary, and assessed milestones keep their dates.
- **A critical path with no float after Week 9.** The chain $F1 \rightarrow H3 \rightarrow H2 \rightarrow H6 \rightarrow I5 \rightarrow$ handover documents (Figure 14) decides the finish. Before 1 October F1 sat in Week 11, after the release work that needs it, an order that could not have finished; after the replan the chain fits, but a slip in F1 delays the release gate unless the contingency in R1 is used. F2, H4 and H1 also feed the gate in Week 10, so they share its zero float (Table 17).
- **Slack lies in capacity, not in calendar time.** Because Week 11 is the last sprint, few tasks have spare weeks; instead Week 11 holds 7.5u against a capacity of 12–28u, and two members carry no counted work in it, which is where a slipped task lands (Section 4.4).

- **Two deliverable-level effects:** the dispute epic finishes in Week 11 rather than Week 10, and evaluation runs earlier than planned, which gives its measurements a week of slack.

4. MILESTONES & REPORTING

4.1 Updated milestones

Table 16 updates the milestone table from the proposal. Following the template's convention, **red text** marks a change and ~~struck-through text~~ marks a task moved or replaced; owners are named for every task: Week 9 owners come from the Jira board, and Weeks 10–12 show proposed owners (*) to be confirmed at the weekly meeting. Ticket-level detail follows in Table 17.

Table 16. Milestones, tasks and reporting, with changes since the proposal

Week	Tasks	Reporting	Status
Weeks 1–3 to 23 Aug	Project kick-off and team introductions; research on the brief; repository and workspace access	Client meeting to review the work plan	Done
Week 4 24–30 Aug	Milestone: Status Report 1 (26 Aug); kick-off and scope agreement; codebase review; technical audit (29 defects, 30 after a later find)	Vision and expectation workshop	Done
Week 5 31 Aug – 6 Sep	Milestone: Proposal. Backlog of 59 stories; pull-request review workflow; schema and contract audit replaced: contracts fixed inside each feature	None	Done
Week 6 7–13 Sep	Validate contracts and schema ; fix cross-project write scoping (G3); added: draft ownership (D6), legacy review writes retired (D2), AI provider integration (C1, C3)	Progress update with client	Done
Week 7 14–23 Sep	Core backend: atomic draft submission (D5); canonical annotation model → per-author versions (F3), moved to the break; task lifecycle → W8; added: PostgreSQL and two-database CI, work-queue API (D8), project policy (B2), AI batch jobs (C2)	Weekly client meeting; written Q&A round 1 (21 Sep); Status Check 2 (23 Sep)	Done; 4 tickets carried
Week 8 24–30 Sep	Milestone: Status Report 2 held on 23 Sep, the last day of sprint W7 (teaching Week 8). Dataset ingestion transactionality (B5); AI module integration (C2); export/completion rule (B6); task lifecycle (B4); added: JSONL import (B7), annotation surfaces (B8), review chooser (D4, D8)	Written Q&A round 2 (24 Sep)	Done; 4 tickets carried
Mid-semester break 1–7 Oct	New sprint: scenario testing (3 bugs fixed and page load cut from 2,969 to ~560 ms); evaluation harness (I1); casebook structure (I2, draft); answer versions (F3); owner reopens a final item (D9); blind second review (D7); disputes (E3/E4) paused on issue #40	Client meeting (7 Oct)	Done, except E3/E4 and I2
Week 9 8–14 Oct	Milestone: Progress Report. Provenance events F1 (Parth), moved from W11; versioned guidelines F2 (Michael); corrections D3 and mode on every item C4 (Jingwei); one authoritative answer H4 (Dishank), from W10 ; release artefact H1 (Kanishka); automatic disputes E1 and B3 check (Yi); repeatable runs I1 (Hanchen); project overview J2 and B1 check (Tim). Human review workflow; role-based governance delivered in W6–break	Progress update with client (14 Oct)	Planned; 2 PRs already in review

Week	Tasks	Reporting	Status
Week 10 15–21 Oct	Release gate H3 (Dishank*) and manifest H2 (Kanishka*); dispute screens E6 (Jingwei); Expert Gate G2 (Yi*); item timeline F5 (Parth*) and actor kinds F4 (Jingwei); casebook to 40 cases I2 (Tim*) and agreement measure I3 (Parth*); AI-influence measures and blind-then-reveal protocol I4 (Michael*); project archive J4 (Tim*); reads never write A6 (Hanchen). Dispute resolution workflow mostly built; screens here	Client meeting to review the logic implemented	Planned
Week 11 22–28 Oct	Last build sprint. Real-data export, policy and history screens H6 (Kanishka*, first, P0); escalation posture E5 (Yi*); integrity checks I5 (Parth*) and findings record (Hanchen*); casebook toward 60 (story owners); export figure check H5 (Dishank*); J1 (Tim*, first to drop). Demo of end-to-end integration end-to-end runs continuously on AG News data; rehearsal after H6	Progress update with client	Planned
Week 12 from 29 Oct	Milestone: Status Report 3 and demo. No build work after 28 Oct. Technical report (Hanchen, Jingwei*), per-member decision records (every member), findings (Hanchen, Parth*) and demonstration rehearsal (Michael*), 29 Oct – 1 Nov; bug review only	Client meeting to showcase the demo	Planned
Week 13	Milestone: Final presentation; final report and technical report submitted	Client meeting for presentation and walkthrough	Planned

4.2 Remaining work, dependencies and the critical path

Figure 14 lays out the remaining stories by week with their dependencies; the blue chain is the critical path, and the orange box is the work blocked on the client. Table 17 gives each ticket's owner, load, float and dependencies. *Float* is how many weeks a ticket can slip without delaying the end of build work on 28 October. Week 9 owners are on the Jira board; Week 10 and 11 owners marked with an asterisk are our proposal, following each member's earlier work, to be confirmed at the weekly meeting.

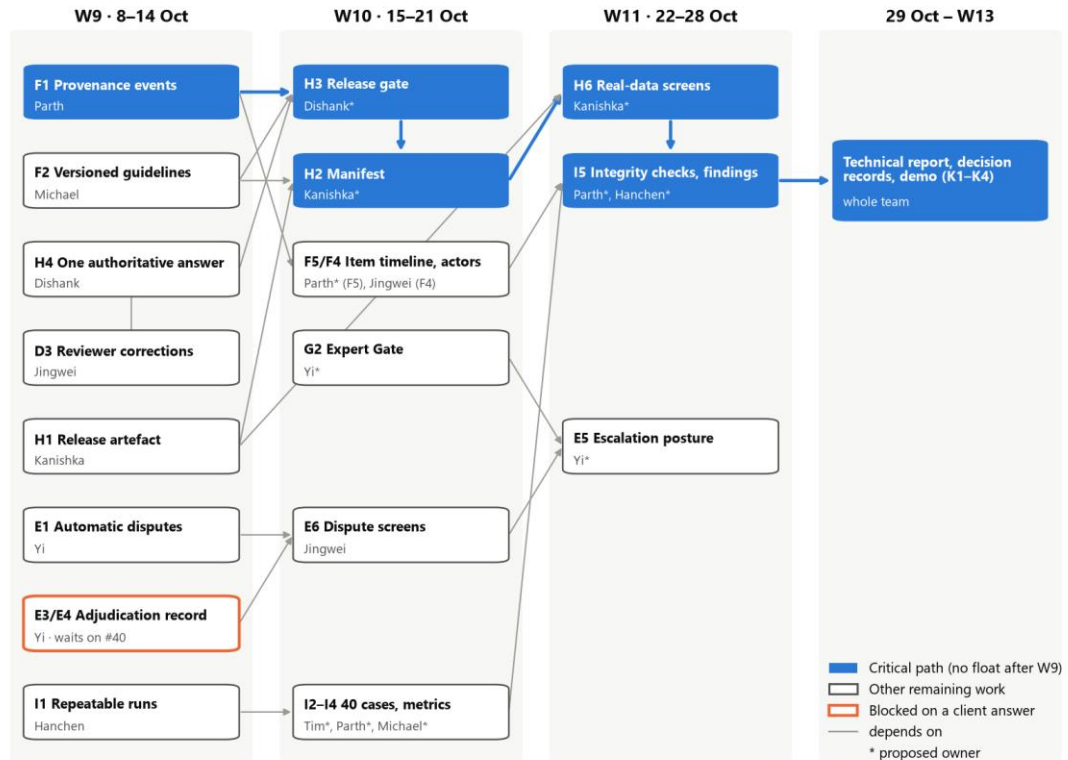


Figure 14. Dependency network of the remaining work, with owners and the critical path

Table 17. Remaining tickets, Weeks 9–11 (CP = critical path; * = proposed owner)

Week	Ticket	Work (story)	Owner	Load	Float	Depends on → unblocks
W9	SCRUM-98	Provenance event record (F1) (CP)	Parth	3u	0	F3 (done) → H3, H2, F5, I5
W9	SCRUM-53	Versioned guidelines, sources, policies (F2)	Michael	3u	0	None → H2, H3
W9	SCRUM-32	Reviewer corrections stored (D3), PR #58	Jingwei	2.5u	0	D4 (done) → H4
W9	SCRUM-37	One authoritative answer per item (H4)	Dishank	2.5u	0	F3, D4 (done); #40 for differing answers → H3
W9	SCRUM-102	Immutable release artefact (H1)	Kanishka	2u	0	D8 list (#49, done) → H2, H6
W9	SCRUM-101	Disagreement opens a dispute (E1)	Yi	1.5u	0	D7 comparison (#53, done) → E6
W9	SCRUM-87	Annotation mode on every item (C4), PR #59	Jingwei	1u	0	None → I4
W9	SCRUM-69	Repeatable runs, regression report (I1)	Hanchen	1.5u	0	Harness (#39, done) → I3, I4
W9	SCRUM-89, 91	Project overview (J2); verify B1	Tim	2u	2 wk	B4, B6 (done)
W9	SCRUM-85	Verify B3	Yi	0.5u	2 wk	None
W9	SCRUM-41	History without repetition (F4), merged early (#57)	Jingwei	0.5u	done	None → SCRUM-40
W10	SCRUM-105	Release gate (H3) (CP)	Dishank*	2u	0	F1, F2, H4 → H2
W10	SCRUM-104	Release manifest (H2) (CP)	Kanishka*	2u	0	H3, H1, F2 → H6
W10	SCRUM-103	Dispute and arbitration screens (E6)	Jingwei	2u	0	E1, E3/E4 → E5
W10	SCRUM-73	AI-influence measures; blind-then-reveal (I4)	Michael*	3u	0	C4, I1 → I5
W10	SCRUM-71	Casebook to 40 cases (I2)	Tim*	2u	0	I1, #51 → I5
W10	SCRUM-72	Reviewer agreement measure (I3)	Parth*	1.5u	0	I1, D7 → I5
W10	SCRUM-106	An item's whole timeline on one screen (F5)	Parth*	1.5u	0	F1, F3, H4 → I5

Week	Ticket	Work (story)	Owner	Load	Float	Depends on → unblocks
W10	SCRUM-40	Actor kinds in the history (F4)	Jingwei	1u	0	F1 → F5, I5
W10	SCRUM-118	Expert Gate; no AI-only finalisation (G2)	Yi*	2u	0	D1 guards (done) → E5
W10	SCRUM-45	Update and archive a project (J4)	Tim* (board: Jingwei)	1.5u	1 wk	B4, H1
W10	SCRUM-121	Reads never write (A6)	Hanchen	2u	1 wk	SCRUM-99's outcome
W11	SCRUM-108	Real-data release, policy, history screens (H6), P0 (CP)	Kanishka*	2u	0	H1, H2 → I5
W11	SCRUM-74	Integrity checks on provenance, roles, releases (I5) (CP)	Parth*	1.5u	0	F4, F5, H1–H4, J3 → K3
W11	SCRUM-75	Findings record with negative results (I5) (CP)	Hanchen*	1u	0	I2–I4, SCRUM-74 → K1, K3
W11	SCRUM-107	Escalation posture per project (E5), P1	Yi*	2u	0	G2, E6
W11	No ticket	Export figure check (H5)	Dishank*	0.5u	0	H1–H4
W11	SCRUM-88	Stable organisation identity (J1), P2	Tim*	0.5u	drop	None

4.3 Who carries what

Table 18 turns the plan into each member's load per week, which is how we check that the timeline is possible: everyone stays within the agreed 1.5–3.5u a week (Jingwei's 4u in Week 9 is by Jingwei's own choice, and two of those tickets are already in review), and the days after build work ends have named owners. Proposed owners are marked with an asterisk as in Table 17.

Table 18. Load per member, Weeks 9–11 and the handover days

Member	W9 (board)	W10	W11	29 Oct – 1 Nov
Hanchen Wang	I1 runs 1.5u	A6 reads 2u	I5 findings 1u*	Technical report lead (K1); findings (K3)*; own decision record
Jingwei Lin	D3 2.5u, C4 1u, F4 0.5u (4u)	E6 2u, F4 actors 1u (3u)	Buffer: carry-over and reviews*	Technical report co-lead*; own decision record
Yi Geng	E1 1.5u, B3 0.5u (2u)	G2 Expert Gate 2u*	E5 posture 2u*	Own decision record
Michael Max	F2 3u	I4 measures 3u*	Buffer: demonstration script on local Ollama*	Demonstration lead (K4)*; own decision record
Parth	F1 3u	F5 1.5u, I3 1.5u (3u)*	I5 integrity 1.5u*	Findings with Hanchen (K3)*; own decision record
Dishank Aswal	H4 2.5u	H3 gate 2u*	H5 check 0.5u*	Own decision record
Kanishka Kathait	H1 2u	H2 manifest 2u*	H6 screens 2u*	Own decision record
Tok Tin (Tim) Chung	J2, B1 2u	I2 cases 2u, J4 1.5u (3.5u)*	J1 0.5u*	Own decision record
Total	20u	20.5u	7.5u (+ buffer)	Whole team (K2)

4.4 Slack, fallbacks and contingency

Figure 15 compares each sprint's planned load with what was completed, against the team's capacity band. Weeks 9 and 10 are planned near the middle of the band;

Week 11 is deliberately light because no sprint follows it, and that spare capacity is the plan's fall-back space. Table 19 names, for each likely slip, the fallback and where the work goes.

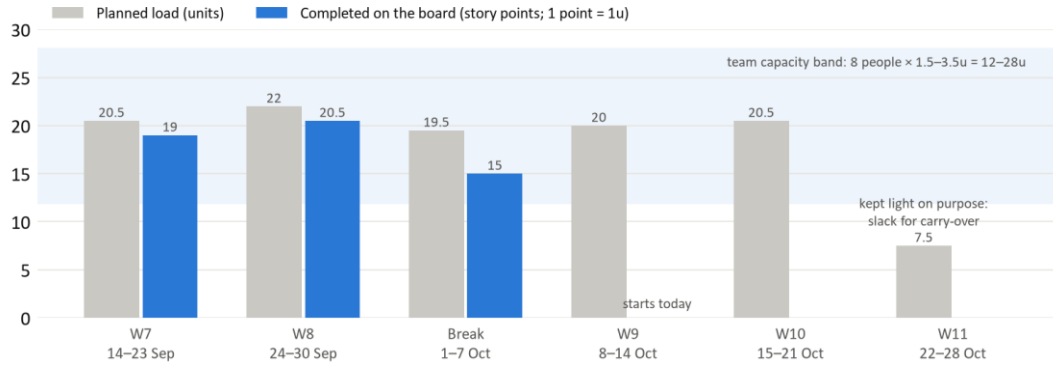


Figure 15. Planned load and completed story points by sprint, with the team's capacity band

Table 19. Contingencies: trigger, fallback and fall-back space

Trigger	Fallback	Fall-back space
F1 not merged when W9 closes (R1)	Build the gate's event-free checks first; add the provenance check when F1 merges	W10 H3 slot; W11 buffer (Jingwei, Michael)
No answer on #40 by the W10 planning meeting (R2)	Written proposal to the client; build it behind a reversible ADR	W10 E6 slot
Any W9-W10 ticket carried over	Carry-over is planned first, then rebalanced so nobody exceeds 3.5u	W11 buffer: 4.5-20.5u of capacity
W11 still overflows	Drop J1 (0.5u), then reduce E5 (P1) by agreement with the client; never P0 or critical-path work	Released capacity goes to H6 and I5
Casebook below 40 cases when W10 closes (R4)	Each story owner adds cases for their own stories	W11 buffer
CI unavailable	Run the full suite locally and attach the result to the PR	Not needed
AI provider fails during the demo (R9)	Run on a local Ollama model or scripted answers	Demonstration rehearsal, 29 Oct – 1 Nov

At every weekly meeting the team applies three replanning rules:

- **Rule 1:** mark finished tickets done, verified against main.
- **Rule 2:** move unfinished tickets into the next week *before* pulling in new work, and rebalance so each person carries 1.5-3.5u.
- **Rule 3:** if a client decision slips, the blocked group takes the fallback named in its roadmap row, pulled forward from a later week, and the blocked story returns at the first meeting after the answer.

4.5 Reporting and communication

- **Weekly client meeting** (Wednesday evening): progress since the last meeting, a demonstration of merged work, and open questions. Week 9's meeting on 14 October reviews F1 and the answers to #38 and #40.
- **Client questions** go to the repository as GitHub issues labelled QA; the issue thread is the record, and affected tickets and ADRs are updated the same day.
- **Internal tracking:** the Jira board and contribution tracker are reconciled with main weekly, and the client-facing backlog page is refreshed after each reconciliation.
- **Assessed milestones ahead:** Status Report 3 with the demonstration (Week 12); the final presentation, final report and technical report (Week 13).

5. ACKNOWLEDGEMENT OF AI USE

[To be completed by the team: state whether generative AI tools were used for this report and for the project, which tools, for what purpose and to what extent.]

APPENDIX A. IDENTIFIERS AND TERMS

Table 20. Identifiers and terms used in this report

Term	Meaning
Story, epic (e.g. D5)	A user story in the backlog, named by its epic letter (A–K) and number. Epics: A foundations, B setting up work, C AI first pass, D review and cross-validation, E disagreement and dispute, F provenance, G roles and permissions, H release and export, I evaluation, J organisations and dashboard, K evidence and handover
SCRUM-n	A ticket on the team's Jira board; a story usually has one to three tickets
#n	A pull request (or issue) in the GitHub repository USYD-CS-Capstone/hej
Unit (u), story point	Planning measure: one unit is two to three subtasks; one Jira story point equals one unit; each person carries 1.5–3.5u a week
P0, P1, P2	Story priority, from must-have (P0) to nice-to-have (P2)
Definition of Done	Demonstrable in the running app; tests in the same change; a fixed defect proven by a test that fails on the old code; documentation updated; reviewed by someone other than the author
Provenance	The recorded history of how a label came to be: source, guideline version, AI suggestion, human response, reviews and adjudication, each with an actor and a time
Authoritative answer	The one version of an item's answer that a release carries; every other version is kept as history
Cross-review	An independent, blind second review of a sampled share of submissions, used to measure agreement
Escalation, dispute, adjudication	An item reviewers cannot settle is escalated; a dispute is the record of that disagreement; adjudication is an expert's final ruling on it
Expert Gate	A project setting that holds an item after its approvals until an independent expert confirms it
Human-first, AI-assisted	Task modes: people annotate first, or the AI gives a first pass that people review
Blind-then-reveal	An evaluation protocol: the annotator answers before seeing the AI's suggestion, then may keep or revise, to measure the AI's influence

Term	Meaning
ADR	Architecture decision record: a short note of a decision, the alternatives and the reasons
Critical path	The chain of dependent work that determines the finish date; a delay on it delays the end